

```
import tensorflow as tf
import numpy as np
import pandas as pd
import cv2 #computer vision
import matplotlib.pyplot as plt
from tensorflow.keras import datasets, layers, models
import random
```

```
(x_train,y_train),(x_test,y_test)= datasets.cifar10.load_data()
```

```
print(x_train.shape)
print(x_test.shape)
```

```
(50000, 32, 32, 3)
(10000, 32, 32, 3)
```

```
#display sample images
class_names= ['airlane','automobile','bird','cat',
              'deer','dog','frog','horse','ship','truck']
```

```
plt.figure(figsize=(10,10))
```

```
for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(x_train[i])
    plt.title(class_names[y_train[i][0]])
    plt.axis('off')
```

```
plt.show()
```

frog



truck



truck



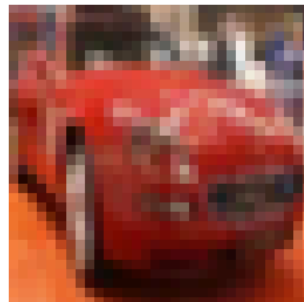
deer



automobile



automobile



bird



horse



ship



```
#normalize images
x_train = x_train/255.0
x_test = x_test/255.0
```

```
#CNN model
model = models.Sequential()
model.add(layers.Conv2D(32,(3,3),
                        activation = 'relu',
                        input_shape=(32,32,3)))
model.add(layers.MaxPooling2D((2,2)))
model.add(layers.Conv2D(64,(3,3),activation = 'relu'))
model.add(layers.MaxPooling2D((2,2)))

model.add(layers.Conv2D(64,(3,3),activation = 'relu'))

#flatten layers
model.add(layers.Flatten())

#dense layer
model.add(layers.Dense(64,activation='relu'))

#output layer

model.add(layers.Dense(10))

#compile model
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])

x_train_small = x_train[:10000]
y_train_small = y_train[:10000]
#train model
history = model.fit(x_train_small,y_train_small,
                    epochs=3,
                    validation_data=(x_test,y_test))

#evaluate model
test_loss,test_acc = model.evaluate(x_test,y_test)
print("Accuracy:",test_acc)

# accuracy graph
plt.plot(history.history['accuracy'],label='accuracy')
plt.plot(history.history['val_accuracy'],label='val_accuracy')

plt.xlabel('epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()

#loss graph
plt.plot(history.history['loss'],label='loss')
plt.plot(history.history['val_loss'],label='val_loss')
plt.xlabel('epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()

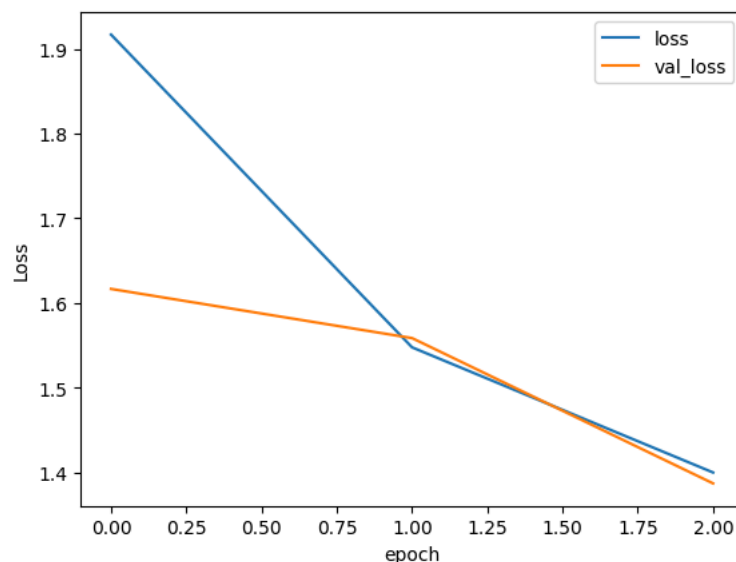
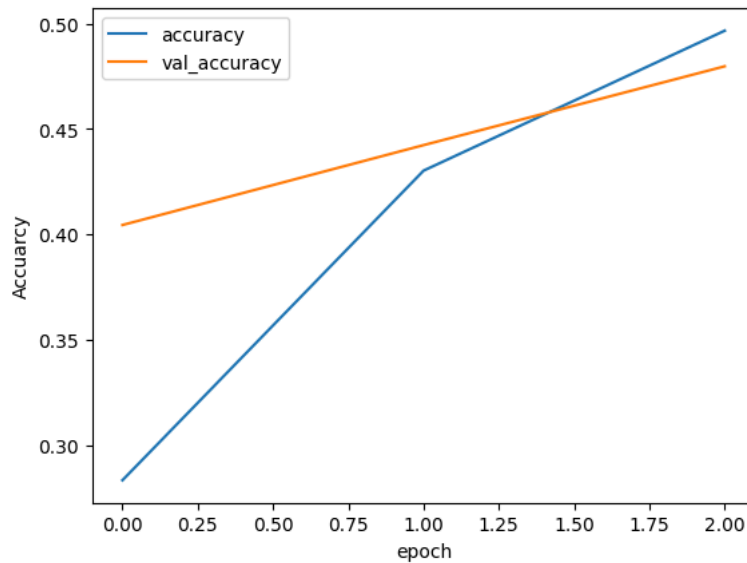
#predictions
predictions = model.predict(x_test)
predicted_labels= np.argmax(predictions, axis = 1)

# Display prediction
plt.imshow(x_test[0])
plt.title("Predicted:"+class_names[predicted_labels[0]])
plt.show()
```

```

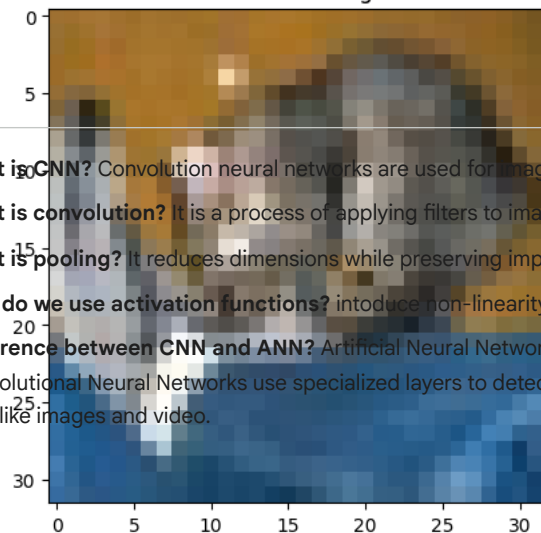
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/3
313/313 ————— 19s 57ms/step - accuracy: 0.2833 - loss: 1.9167 - val_accuracy: 0.4044 - val_loss: 1.6165
Epoch 2/3
313/313 ————— 18s 59ms/step - accuracy: 0.4302 - loss: 1.5475 - val_accuracy: 0.4423 - val_loss: 1.5584
Epoch 3/3
313/313 ————— 18s 58ms/step - accuracy: 0.4966 - loss: 1.3998 - val_accuracy: 0.4797 - val_loss: 1.3869
313/313 ————— 4s 12ms/step - accuracy: 0.4797 - loss: 1.3869
Accuracy: 0.4796999990940094

```



313/313 ————— 4s 11ms/step

Predicted:frog



What is CNN? Convolution neural networks are used for image processing and classification in deep learning algorithm

What is convolution? It is a process of applying filters to images to extract features

What is pooling? It reduces dimensions while preserving important features

Why do we use activation functions? introduce non-linearity so the model can learn complex patterns.

Difference between CNN and ANN? Artificial Neural Networks are foundational, fully connected networks best for structured data. Convolutional Neural Networks use specialized layers to detect spatial patterns, making them the industry standard for unstructured data like images and video.